



SERVICIO NACIONAL DE APRENDIZAJE - SENA

Analisis y Desarrollo de Software

TALLER SOBRE COMPONENTES FRONT-END

Evidencia GA7-220501096-AA4-EV01

Proyecto de referencia

SAAAF - Sistema de Asignacion de Actas de Activos Fijos. El taller relaciona los conceptos de React con los modulos y flujos que se han trabajado para el front-end del proyecto.

Aprendiz	Brayan Giraldo Garcia
Ficha	3235876
Programa	Analisis y Desarrollo de Software
Proyecto formativo	Construccion de software integrador de tecnologias orientadas a servicios
Fecha	2026

Introduccion

El desarrollo del front-end de una aplicacion moderna requiere comprender como se construyen las interfaces, como se organizan sus componentes y como se responde a las acciones de las personas usuarias. React permite abordar estos aspectos mediante una arquitectura basada en componentes, una sintaxis declarativa y mecanismos para administrar datos e interacciones. Aunque React se define tecnicamente como una biblioteca de interfaces, en el proyecto SAAAF se relaciona con Next.js, que aporta la estructura de framework para organizar rutas, pantallas y funcionalidades del front-end.

El presente taller desarrolla los conceptos solicitados en la evidencia GA7-220501096-AA4-EV01 y los contextualiza en SAAAF, sistema orientado a la gestion de activos fijos, empleados, areas, asignaciones, actas y devoluciones. De esta manera, los conceptos de React no se presentan de forma aislada, sino asociados a situaciones concretas del proyecto trabajado durante la fase de ejecucion.

Objetivos

Objetivo general

Analizar los principales conceptos de React y JSX y relacionarlos con la construccion del front-end del proyecto SAAAF, identificando componentes, clases, eventos y elementos necesarios para desarrollar interfaces interactivas y reutilizables.

Objetivos especificos

- Diferenciar React de JSX mediante una tabla comparativa aplicada al contexto de SAAAF.
- Explicar el concepto de clases en React y su relacion con los componentes y el manejo de estado.
- Identificar los principales eventos de React y ejemplificar su uso en formularios y acciones del sistema SAAAF.
- Construir un mapa conceptual que integre React, JSX, componentes, props, estado, eventos, hooks, DOM virtual y Next.js.

Caracteristicas principales del front-end con React

Arquitectura por componentes Separa Dashboard, Inventario, Asignaciones, Empleados, Areas y Actas en piezas reutilizables.	Enfoque declarativo La pantalla refleja el estado actual de los datos sin manipular manualmente cada cambio visual.
JSX Expresa de forma legible tarjetas, formularios, tablas y botones dentro de JavaScript.	Props y estado Comunican datos entre componentes y conservan valores que cambian durante la interaccion.
Manejo de eventos Conecta clics, cambios de campos y envios con acciones como guardar, asignar o devolver.	DOM virtual Actualiza de manera eficiente las partes de la interfaz que realmente cambian.

Integración con Next.js

Aporta estructura de framework, rutas y organización para el front-end de SAAAF.

1. Diferencia entre React y JSX

React y JSX se utilizan juntos con frecuencia, pero no representan lo mismo. React proporciona el modelo de componentes, estado, propiedades y actualización de la interfaz; JSX es una sintaxis que facilita describir visualmente lo que un componente debe renderizar. En un front-end de SAAAF, React administraría la lógica de una pantalla como Inventario, mientras que JSX permitiría escribir de forma legible la tabla, los botones y los campos que aparecen en esa pantalla.

Aspecto	React	JSX
Definición	Biblioteca de JavaScript para construir interfaces basadas en componentes.	Extensión de sintaxis de JavaScript que permite escribir estructuras similares a HTML dentro del código.
Función principal	Administrar componentes, propiedades, estado, renderizado y actualización de la interfaz.	Describir de manera clara la estructura visual que un componente React debe mostrar.
Ejecución	React participa en la creación y actualización de la interfaz.	JSX se transforma a llamadas de JavaScript antes de ejecutarse en el navegador.
Uso en SAAAF	Permite construir módulos como Dashboard, Activos, Empleados, Áreas, Asignaciones y Actas.	Permite expresar tarjetas, tablas, formularios, botones y mensajes de esos módulos.
Datos dinámicos	Gestiona cambios mediante props, estado y hooks.	Inserta expresiones JavaScript con llaves, por ejemplo {activo.serial} o {empleado.nombre}.
Eventos	Define cómo se procesan acciones del usuario mediante manejadores.	Permite asociar dichos manejadores a elementos, por ejemplo onClick={generarActa}.
Relación	Puede utilizarse sin JSX, aunque sería menos legible.	Necesita ser transformado a JavaScript y normalmente se utiliza dentro de componentes React.

Ejemplo aplicado a SAAAF

```
function TarjetaActivo({ activo }) {  
  return (  
    <article className="tarjeta-activo">  
      <h3>{activo.nombre}</h3>  
      <p>Serial: {activo.serial}</p>  
      <p>Estado: {activo.estado}</p>  
    </article>  
  );  
}
```

```
}
```

En el ejemplo, React representa el componente `TarjetaActivo` y la forma en que recibe la prop `activo`. JSX corresponde al contenido incluido dentro del `return`, donde se mezclan etiquetas visuales con expresiones JavaScript como `{activo.serial}`.

2. ¿Que son clases en React?

Las clases en React son una forma tradicional de crear componentes mediante clases de JavaScript que extienden `React.Component`. Un componente de clase dispone de un metodo `render()` para devolver la interfaz y puede mantener informacion interna mediante `state`. Tambien puede implementar metodos del ciclo de vida para ejecutar acciones en diferentes momentos del componente.

Uso actual

En desarrollos modernos se prefieren los componentes funcionales y los hooks, porque permiten expresar la misma logica con menos codigo. Sin embargo, comprender las clases sigue siendo importante para leer proyectos existentes y entender conceptos como estado, propiedades y ciclo de vida.

Dentro del dominio SAAAF, una clase podria representar una lista de activos que conserva en su estado los registros que se muestran. El siguiente ejemplo es didactico y muestra el concepto solicitado por la evidencia:

```
class ListaActivos extends React.Component {
  constructor(props) {
    super(props);
    this.state = { activos: [] };
  }

  componentDidMount() {
    // Aqui se podria consultar la API de activos de SAAAF.
  }

  render() {
    return (
      <section>
        <h2>Inventario de activos</h2>
        <p>Total: {this.state.activos.length}</p>
      </section>
    );
  }
}
```

Elemento	Funcion	Relacion con SAAAF
React.Component	Clase base de la que hereda el componente.	Permite encapsular una pantalla o una seccion de la interfaz.
props	Datos recibidos desde un componente padre.	Por ejemplo, el activo o empleado seleccionado.
state	Datos internos que pueden cambiar.	Lista de activos, filtros, mensajes o seleccion actual.
render()	Devuelve la estructura visual del componente.	Construye la tabla o formulario que ve la persona usuaria.
componentDidMount()	Se ejecuta despues de montar el componente.	Puede usarse para consultar datos iniciales del sistema.

3. Principales eventos de React

React permite responder a las interacciones mediante propiedades de eventos escritas en camelCase. En lugar de colocar texto de JavaScript dentro de un atributo HTML, se pasa una funcion manejadora. React entrega a esa funcion un objeto de evento que permite consultar informacion de la accion y, cuando se necesita, impedir el comportamiento predeterminado del navegador.

Evento	Uso	Ejemplo en SAAAF
onClick	Clic sobre botones o elementos interactivos.	Boton Generar acta, Editar activo, Tomar asignacion o Registrar devolucion.
onChange	Cambio en el valor de un campo.	Actualizar serial, placa, empleado, area, observaciones o filtros de busqueda.
onSubmit	Envio de un formulario.	Guardar un activo, crear una asignacion o registrar una devolucion.
onFocus	Un control recibe el foco.	Mostrar una ayuda contextual al ingresar al campo Serial o Responsable.
onBlur	Un control pierde el foco.	Validar que un serial o placa tenga el formato esperado antes de continuar.
onKeyDown	Se presiona una tecla.	Permitir busquedas con Enter y mejorar la navegacion por teclado.
onDoubleClick	Doble clic sobre un elemento.	Abrir rapidamente el detalle de un activo, si el diseno de interfaz lo contempla.
onMouseEnter / onMouseLeave	Entrada o salida del puntero.	Mostrar ayuda visual o resaltar acciones sin alterar los datos.

Ejemplo: formulario de asignacion

```
function FormularioAsignacion() {
  const [empleado, setEmpleado] = React.useState("");

  const guardarAsignacion = (event) => {
    event.preventDefault();
    // Validar datos y enviar la asignacion al back-end de SAAAF.
  };

  return (
    <form onSubmit={guardarAsignacion}>
      <input
        value={empleado}
        onChange={(event) => setEmpleado(event.target.value)}
        onBlur={() => console.log('Validar empleado')}
      />
      <button type="submit">Asignar activo</button>
    </form>
  );
}
```

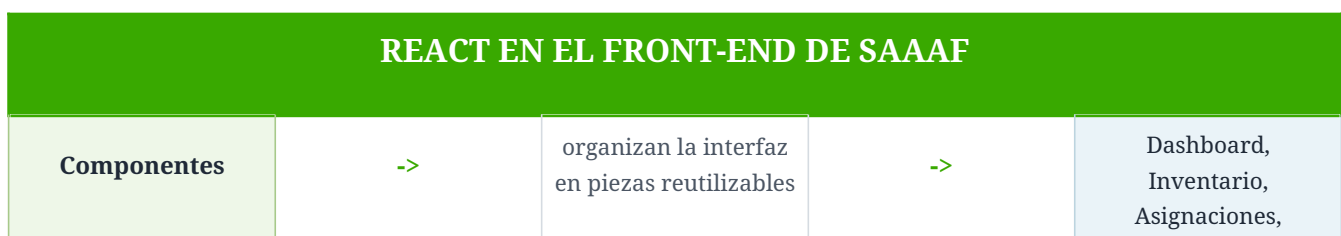
El evento `onChange` mantiene sincronizado el campo con el estado del componente. `onBlur` puede activar una validacion cuando la persona sale del campo y `onSubmit` concentra la accion de guardar. `event.preventDefault()` evita que el navegador recargue la pagina al enviar el formulario.

Eventos y buenas practicas para SAAAF

- Nombrar los manejadores de manera descriptiva: `guardarActivo`, `generarActa`, `registrarDevolucion` o `aplicarFiltro`.
- Mantener la validacion separada de la presentacion cuando la logica sea compleja.
- Deshabilitar temporalmente botones de envio mientras se procesa una solicitud para evitar registros duplicados.
- Mostrar mensajes claros de exito o error despues de las operaciones.
- Conservar accesibilidad de teclado y etiquetas de formulario adecuadas.

4. Mapa conceptual de React aplicado a SAAAF

El siguiente mapa integra los conceptos principales del componente formativo y los vincula con funcionalidades del proyecto SAAAF. La lectura se realiza desde React hacia cada concepto, su relacion y una aplicacion concreta dentro del sistema.



				Empleados, Areas y Actas
JSX	->	describe la interfaz combinando JavaScript y marcado	->	formularios, tarjetas, tablas y botones del sistema
Props	->	transportan datos entre componentes	->	activo seleccionado, empleado, area o datos de una acta
Estado	->	mantiene datos que cambian durante la interaccion	->	filtros, formularios, lista de activos, mensajes y estados de carga
Eventos	->	responden a acciones de la persona usuaria	->	guardar asignacion, generar acta, editar, devolver o buscar
Hooks	->	gestionan estado y efectos en componentes funcionales	->	useState para formularios; useEffect para consultar datos
Next.js	->	aporta estructura de framework sobre React	->	rutas, pantallas y organizacion del front-end del proyecto SAAAF
DOM virtual	->	actualiza de forma eficiente solo lo necesario	->	refresca tablas, tarjetas y formularios sin recargar toda la pagina

Lectura del mapa conceptual

React organiza la interfaz mediante componentes. Estos componentes pueden describir su estructura con JSX, recibir informacion a traves de props y conservar datos variables mediante estado. Los eventos capturan las acciones de la persona usuaria y los hooks permiten gestionar estado y efectos en componentes funcionales. El DOM virtual facilita actualizaciones eficientes y Next.js aporta al proyecto una estructura de framework para organizar el front-end. En SAAAF, esta relacion se refleja en pantallas como Inventario, Asignaciones, Empleados, Areas y Actas.

Aplicacion de los conceptos al proyecto SAAAF

Los conceptos del taller pueden trasladarse directamente al diseño del front-end de SAAAF. Una organizacion por componentes ayuda a mantener el sistema modular y facilita que cada funcionalidad se pueda probar y modificar sin afectar innecesariamente a las demas.

Modulo	Interfaz	Conceptos React	Eventos principales
Dashboard	Tarjetas, indicadores, ultimos movimientos.	Props, estado, JSX, useEffect.	onClick para navegar; carga de datos al iniciar.
Inventario	Tabla de activos, filtros, formulario de alta y edicion.	Estado, listas, formularios controlados.	onChange, onSubmit, onClick, onBlur.
Asignaciones	Selección de activo y empleado, generacion de asignacion.	Componentes reutilizables, props y estado.	onChange y onSubmit.
Empleados y Areas	Catalogos para asociar responsables y ubicaciones.	Componentes de lista y formulario.	onClick, onChange, onSubmit.
Actas	Detalle, generacion y consulta de actas.	Props para datos del acta y renderizado condicional.	onClick para generar, descargar o consultar.
Devolucion	Registro del retorno del activo y cambio de estado.	Estado del formulario y validaciones.	onSubmit y onBlur.

Ejemplo de componente funcional recomendado

Para nuevas pantallas de SAAAF resulta apropiado utilizar componentes funcionales con hooks. El siguiente ejemplo simplificado muestra una busqueda de activos y relaciona JSX, estado y eventos en una misma pieza de interfaz:

```
function BuscadorActivos({ activos }) {  
  const [texto, setTexto] = React.useState("");  
  
  const filtrados = activos.filter((activo) =>  
    activo.serial.toLowerCase().includes(texto.toLowerCase())  
  );  
  
  return (  
    <section>  
      <input  
        value={texto}  
        onChange={(event) => setTexto(event.target.value)}  
        placeholder="Buscar por serial"  
      />  
      <p>Resultados: {filtrados.length}</p>  
    </section>  
  );  
}
```

Trazabilidad con los criterios de evaluacion

No.	Criterio	Evidencia en el documento
1	Realiza una tabla de diferencias indicada en el punto 1.	Seccion 1: tabla comparativa entre React y JSX.
2	Presenta una definicion coherente al punto 2.	Seccion 2: definicion, estructura y ejemplo de componentes de clase.
3	Tiene claridad sobre los principales eventos que se utilizan en React.	Seccion 3: tabla de eventos, ejemplo de formulario y buenas practicas.
4	Realiza el mapa conceptual aplicando todos los conceptos vistos.	Seccion 4: mapa conceptual de React y aplicacion directa a SAAAF.

Conclusiones

- React permite estructurar interfaces en componentes reutilizables y facilita que los cambios de datos se reflejen de forma declarativa en la pantalla.
- JSX no reemplaza a React; funciona como una sintaxis que hace mas clara la descripcion de los elementos visuales que React debe renderizar.
- Los componentes de clase permiten comprender conceptos historicos y fundamentales como state, props y ciclo de vida, aunque en desarrollo moderno predominan los componentes funcionales con hooks.
- Los eventos son esenciales en SAAAF porque conectan acciones como registrar activos, asignar responsables, generar actas y procesar devoluciones con la logica de la aplicacion.
- La combinacion React y Next.js es adecuada para organizar un front-end modular de SAAAF, manteniendo separadas las responsabilidades de cada pantalla y facilitando su evolucion.

Referencias

React. Documentacion oficial: Learn React. <https://react.dev/learn>

React. Documentacion oficial: Responding to Events. <https://react.dev/learn/responding-to-events>

React. Documentacion oficial: Component. <https://react.dev/reference/react/Component>

Next.js. Documentacion oficial. <https://nextjs.org/docs>

Servicio Nacional de Aprendizaje - SENA. Material del componente formativo correspondiente a componentes front-end.

Nota de contexto del proyecto

Los ejemplos de codigo incluidos son didacticos y utilizan las entidades y flujos de SAAAF para evidenciar la aplicacion de los conceptos del taller. No se presentan como una copia literal de todos los archivos productivos del proyecto.